المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

Nom & prenom : Meziane Sofyan

IACS S2

TP2 : Monolithe Modulaire :

Restructuration et Préparation a la Migration Microservices :

1- Introduction :

Ce rapport presente la realisation du TP2 qui consiste a restructurer le monolithe simple du TP1 en un monolithe modulaire. L'objectif est de definir des boundaries claires entre domaines, d'implementer des interfaces de communication entre modules et d'appliquer les design patterns (DTO, Mapper, Service Layer) pour preparer une future migration vers une architecture microservices.

2- Objectifs de ce TP :

- ➔ Restructurer le monolithe en modules metier distincts
- ➔ Definir des boundaries claires entre domaines (Product, Customer, Order)
- ➔ Implementer des interfaces de communication entre modules
- ➔ Appliquer les design patterns : DTO, Mapper, Service Layer
- ➔ Utiliser MapStruct pour le mapping DTO <-> Entity
- ➔ Preparer le terrain pour une future migration microservices

3- Les outils Utilisées :

Outil / Technologie	Version	Role
Java / JDK	17+	Langage de programmation
Spring Boot	3.x	Framework principal
Spring Data JPA	—	Acces base de donnees
MapStruct	1.5.x	Mapping automatique DTO <-> Entity
Lombok	—	Reduction du code boilerplate
PostgreSQL	15+	Base de donnees relationnelle
Maven	3.8+	Gestion des dependances
Postman	—	Test des endpoints REST
IntelliJ IDEA	—	Environnement de developpement

4- Structure du TP :

```
com.ecommerce.monolith
|
+-- product
|   +-- controller    -> ProductController.java
|   +-- service       -> ProductService.java / ProductServiceImpl.java
|   +-- repository    -> ProductRepository.java
|   +-- model         -> Product.java
|   +-- dto           -> ProductDTO.java / ProductRequest.java
|   +-- mapper        -> ProductMapper.java
|
+-- customer
|   +-- controller    -> CustomerController.java
|   +-- service       -> CustomerService.java / CustomerServiceImpl.java
|   +-- repository    -> CustomerRepository.java
|   +-- model         -> Customer.java
|   +-- dto           -> CustomerDTO.java / CustomerRequest.java
|   +-- mapper        -> CustomerMapper.java
|
+-- order
    +-- controller    -> OrderController.java
    +-- service       -> OrderService.java / OrderServiceImpl.java
    +-- repository    -> OrderRepository.java
    +-- model         -> Order.java
    +-- dto           -> OrderDTO.java / OrderRequest.java
           +-- mapper        -> OrderMapper.java
```

5- TP1 → TP2 : Évolutions principales :

- **Structure du projet :**
  - **TP1 :** seul module (Product)
  - **TP2 :** modules distincts (Product + Customer + Order)
- **DTOs ajoutés :**

- **TP1 :** Entités directement dans les réponses API
- **TP2 :** DTOs séparés (ProductDTO, CreateProductRequest)

- **MapStruct ajouté :**

- **TP1 :** Pas de mapping
- **TP2 :** Mapping automatique Entity ↔ DTO via MapStruct

- **Service en interface :**

- **TP1 :** Classe concrète directement
- **TP2 :** Interface + Implementation séparées

- **Communication entre modules :**
- **TP1 :** Pas de communication entre modules
- **TP2 :** Order dépend de Product et Customer via services publics
- **Préparation microservices :**
- **TP1 :** Monolithe simple couplé
- **TP2 :** prêt pour migration microservices

6-Les endpoints de l'API REST :

Methode	URL	Description
GET	/api/products	Lister tous les produits
POST	/api/products	Creer un produit
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit
GET	/api/customers	Lister tous les clients
POST	/api/customers	Creer un client
GET	/api/customers/{id}	Obtenir un client
GET	/api/orders	Lister toutes les commandes
POST	/api/orders	Creer une commande
GET	/api/orders/customer/{id}	Historique commandes client

7- Conclusion :

Ce TP2 nous a permis de comprendre l'importance de la modularisation dans le developpement logiciel. En restructurant le monolithe simple du TP1 en un monolithe modulaire, nous avons applique des principes fondamentaux de genie logiciel : separation des responsabilites, couplage faible et cohesion forte.

L'utilisation de MapStruct a simplifie considerablement le mapping entre entites et DTOs, eliminant le code repetitif et reduisant les risques d'erreurs. Les interfaces de service ont rendu le code plus testable et plus facile a maintenir.

Cette architecture modulaire constitue une excellente preparation pour une future migration vers une architecture microservices, ou chaque module deviendra un service independant deployable separement.